

XM · CLASE 5



Soluciones de Analítica con Databricks

Clase 5: Estructura de proyectos, calidad y pipelines declarativos

Docente: Manuela Larrea Gómez

Agenda de la clase 5



Primero la pregunta de la semana pasada; luego, qué es un dato válido y quién lo decide

01

15 MIN

Estructura de un producto de datos

Notebook vs. src · carpeta por carpeta y cuándo hace falta · lo que recomienda Databricks · estándares de industria · GitHub Actions vs. Azure DevOps

02

30 MIN

Calidad y pipelines declarativos

Reglas con acción (warn / drop / cuarentena / fail) · expectations · cuarentena y métricas · sector: pérdidas y balance de energía · casos

03

10 MIN

Demo: el pipeline en dev

Un solo pipeline activo en Free Edition: lo corre la docente. DAG, calidad por tabla, event log

04

55 MIN + 10

Lab 5: validador por lotes en tu esquema

REGLAS en src · evaluar() · cinco filas malas · cuarentena · balance · calidad_metricas · cierre y tag s05

ESTRUCTURA

¿Siempre tengo que tener src?

No. La respuesta larga son las próximas seis diapositivas

Notebook es interfaz; src es producto

La respuesta completa a la pregunta de la clase pasada



Notebook: interfaz al dato

Preguntas y respuestas, en el orden que se te ocurra. Excelente para explorar, demostrar y depurar. No se puede importar, no se prueba sin ejecutarlo entero, y el orden de las celdas es un estado oculto.



src: funciones que reciben y devuelven

Se importan desde cualquier notebook o job, se prueban con pytest (sin Spark si son puras), se despliegan con un bundle. Lo viste en el lab 4: bronze.py tenía la lógica; el notebook la llamaba.

Señal	Qué hacer
Copiaste la celda por segunda vez · alguien más la necesita · la va a correr un job	Se convierte en una función en src, con test
Estás mirando una tabla para entender algo · una gráfica · una prueba de una idea	Se queda en el notebook. Es exploración y no necesita ceremonia

La estructura, carpeta por carpeta, y cuándo hace falta cada una

Nuestro repo sigue la plantilla default-python de los bundles de Databricks, con nombres en español

```
Databricks-to-XM/
├─ databricks.yml    bundle: dev/qa/prod
├─ resources/        jobs y pipelines
├─ conf/             config por ambiente
├─ src/xm_demanda/   el paquete
│   ├─ ingest/       quality/ features/
│   └─ model/        utils/
├─ notebooks/        delgados, 1 por tarea
├─ tests/unit/        pytest sobre src
├─ app/              Databricks App
├─ docs/ adr/         arquitectura, ADRs
├─ data/raw           dataset del caso
├─ .github/workflows CI (Actions)
├─ ci/               Azure DevOps (ref.)
└─ clases/           solo del curso
```

Carpeta	Hace falta cuando...
src + tests	Hay lógica que se reutiliza o que corre sin una persona
notebooks	Siempre; delgados: parámetros → llamar a src → mostrar
conf	Hay más de un ambiente (dev / qa / prod)
resources + databricks.yml	Algo se despliega: un job, un pipeline, un endpoint
docs + adr	Desde el día 1: un ADR de 3 párrafos es más barato que una reunión en 6 meses
app	Alguien consume el resultado sin abrir Databricks
ci / .github	Hay más de una persona o más de un ambiente

Lo que recomienda Databricks (y de dónde sale nuestro repo)

Guía 'Software engineering best practices for notebooks' y plantillas de bundles



Módulos fuera del notebook

"Move some of the code in your notebook into a set of shared functions outside of your notebook." Reutilizable y probable.



Ramas y PR

Trabajar en una rama (eda, first_modules...) y mezclar a main por pull request. Lo que hacen cada clase.



Tests sin ejecutar el notebook

"If the shared code fails to run, the notebook itself would likely fail to run as well." pytest en CI, en cada PR.



Jobs desde Git, no el botón schedule

Un job que apunta a una rama corre la versión committeada. El schedule del notebook corre lo que haya en pantalla.

databricks bundle init ...	Genera
default-python	databricks.yml · resources/ (job + pipeline) · src/ · tests/ — la base de nuestro repo
mlops-stacks	Lo anterior + entrenamiento, registro en UC, promoción dev → staging → prod y CI/CD (clase 13)

Fuente: docs.databricks.com/aws/en/notebooks/best-practices · docs.databricks.com/aws/en/dev-tools/bundles/templates

Estándares de la industria: tres referencias y qué tomamos de cada una

Coinciden en separar exploración de producto, sacar la configuración del código y probar. Difieren en cuánta ceremonia

Referencia	Qué propone	Cuándo encaja	Qué tomamos
Cookiecutter Data Science (DrivenData)	data/ · notebooks/ · src/ · models/ · reports/; notebooks numerados con iniciales del autor	Equipos de ciencia de datos; proyectos de análisis y modelos	notebooks delgados y numerados; src como paquete instalable
Kedro (QuantumBlack / McKinsey)	Pipelines como grafos de nodos; catálogo de datos declarativo; capas raw → intermediate → primary → feature → model	Muchos equipos, muchos pipelines, necesidad de convención fuerte	La idea de capas con contrato (nuestro Medallion) y conf/ por ambiente
MLOps Stacks (Databricks)	Bundle con entrenamiento, registro en UC, validación y despliegue por ambiente; CI/CD listo	Productos con modelo en producción	resources/ para train e inferencia; promoción dev → qa → prod (clase 13)

Se elige por el tamaño del equipo y por lo que va a correr sin ustedes, no por moda.

Fuente: cookiecutter-data-science.drivendata.org · kedro.org · docs.databricks.com/aws/en/machine-learning/mlops/mlops-stacks

GitHub Actions o Azure DevOps: la decisión es organizacional

Los dos hacen lo mismo: tests en cada PR y despliegue del bundle por ambiente. Databricks documenta ambos

	GitHub Actions	Azure DevOps Pipelines
Elige esto si...	El código vive en GitHub; equipos pequeños; open source	El código vive en Azure Repos; identidades y aprobaciones en Entra ID; PMO con Boards
Identidad hacia Databricks	Secrets del repo + service principal (OIDC posible)	Service connection con service principal de Entra ID
Ambientes y aprobaciones	Environments con required reviewers	Environments + approvals; integrados con Boards
En el repo	.github/workflows/ci.yml (el que corre)	ci/azure-pipelines.yml (equivalente, para leer)
Lo que no cambia	ruff · pytest · databricks bundle validate · bundle deploy -t <ambiente>	Idéntico

Regla: donde está el código y quién administra las identidades. Mezclar es posible; duele.

Fuente: docs.databricks.com/aws/en/dev-tools/ci-cd/ · learn.microsoft.com/azure/databricks/dev-tools/ci-cd/

Cuándo sí y cuándo no: la tabla de decisión

La señal más confiable: ¿alguien paga si esto falla un martes a las 6 a. m.?

Qué es	Estructura	Ejemplo en XM
Un análisis de una vez para responder una pregunta	Un notebook. Y ya.	¿Cuánto cayó la demanda del regulado en Semana Santa?
Prototipo de días que puede morir	Notebook + un archivo de funciones al lado. Sin bundle, sin conf	Probar si un modelo por serie mejora al global
Lo corre un job, lo usa otra persona o lo audita alguien	src + tests + conf + notebooks delgados + docs/adr	El pipeline bronce → plata de hoy
Tiene modelo en producción	Lo anterior + resources (train, inferencia, monitor) + registro en UC + CI/CD	pronostico_demanda (clases 9–15)
Varios equipos, varios productos	Lo anterior + plantilla propia (bundle init desde un template de XM) + catálogo por dominio	Demanda, precios, restricciones, cada uno un repo

Empezar con un notebook no es un error. Dejar en un notebook lo que ya corre solo, sí.

CALIDAD

Bronce guarda todo. Plata decide qué sirve

Una regla = una expresión SQL + una acción cuando falla

Una regla es una expresión SQL con una acción

Calidad de datos, en la práctica, es una lista de reglas en un solo sitio



warn

La fila pasa. Se cuenta. Para lo raro que no invalida el dato.



drop

La fila no pasa. Se cuenta. Para lo que no sirve para nada.



cuarentena

La fila va a una tabla aparte con el nombre de la regla. Alguien la mira.



fail

Nada se escribe. El archivo está corrupto o el contrato cambió.

```
# src/xm_demanda/quality/rules.py – la única definición de qué es válido
REGLAS = {
    "valor_no_nulo": ("Valor IS NOT NULL", "drop"),
    "valor_no_negativo": ("Valor >= 0", "cuarentena"),
    "regulado_sin_clasificar": ("TipoMercado <> 'Regulado' OR ClasificacionIndustrial = 'SIN CLASIFICAR'", "warn"),
    "fecha_no_futura": ("Fecha <= current_date()", "fail"),
}
```

Fuente: La misma lista la usa el pipeline declarativo (02_silver_pipeline.py) y el validador por lotes del lab (validador.py)

Nuestras reglas, sobre nuestros datos

Hoy ninguna falla sobre el dato real. En el lab inyectan cinco filas malas para ver a dónde va cada una

Regla	Expresión	Acción	Por qué esa acción
valor_no_nulo	Valor IS NOT NULL	drop	Una fila sin valor no sirve para nada
valor_no_negativo	Valor >= 0	cuarentena	Puede ser error de medida o de signo: alguien lo mira
variable_conocida	CodigoVariable IN ('DdaReal', 'PerdidasEnergia')	drop	Solo aceptamos las dos variables del contrato
regulado_sin_clasificar	TipoMercado <> 'Regulado' OR ClasificacionIndustrial = 'SIN CLASIFICAR'	warn	En el regulado el CIU siempre viene sin clasificar; si no, es raro, no inválido
fecha_no_futura	Fecha <= current_date()	fail	Fechas del futuro = archivo corrupto; no queremos nada de él
perdidas_coherentes (balance)	perdidas_kwh <= demanda_real_kwh, por serie-día	cuarentena	Regla del sector; se evalúa sobre la vista ancha, no fila por fila

Sobre 145.408 filas reales: 0 fallas en las seis. Un dato limpio también se documenta.

Pipelines declarativos: decir qué tablas existen, no cómo construirlas

El sistema deduce el orden, paraleliza, reintenta y guarda métricas por tabla en un log de eventos



Vista materializada

Se recalcula (completa o incrementalmente) a partir de sus fuentes. Para transformaciones por lotes: plata, oro.



Tabla streaming

Procesa solo lo nuevo, con checkpoint. Para ingesta: bronce con Auto Loader dentro del pipeline.



DAG deducido

Cada tabla es una consulta sobre otras. El pipeline lee las dependencias y decide el orden. Nadie escribe 'primero esto, luego aquello'.

Cuándo pipeline declarativo	Cuándo job clásico (tarea de notebook)
Tablas a partir de tablas, con reglas de calidad y dependencias: bronce → plata → oro	Entrenar un modelo, llamar una API, generar un reporte, cualquier cosa que no sea 'tabla a partir de tablas'
En nuestro DAG (clase 12): ingesta y plata	En nuestro DAG: features, entrenamiento, inferencia, monitoreo

Fuente: docs.databricks.com/aws/en/ldp/ · docs.databricks.com/aws/en/ldp/best-practices

Expectations en código: las reglas de src se vuelven decoradores

El pipeline no repite las reglas; las importa. Para eso existe la carpeta src

```
notebooks/02_silver_pipeline.py (extracto)
from pyspark import pipelines as dp
from xm_demanda.quality.rules import REGLAS

def con_expectations(fn):
    for nombre, (expr, accion) in REGLAS.items():
        deco = {"warn": dp.expect,
                "drop": dp.expect_or_drop,
                "fail": dp.expect_or_fail}.get(accion)
        if deco: fn = deco(nombre, expr)(fn)
    return fn

@dp.table(name="demanda_validada")
@con_expectations
def demanda_validada():
    return spark.read.table("LIVE.demanda_vigente")
```

Decorador	Acción
@dp.expect(n, expr)	warn: escribe y cuenta
@dp.expect_or_drop(n, expr)	drop: descarta y cuenta
@dp.expect_or_fail(n, expr)	fail: detiene la actualización
@dp.expect_all_or_drop({...})	Varias reglas en un diccionario

cuarentena no es un decorador

Es un patrón: columna_fallas + dos tablas (limpias / con fallas). Siguiendo la siguiente diapositiva.

Cuarentena y métricas: lo que alguien debe mirar, y cuántas veces pasó

Ambas tablas salen del mismo pipeline; la de cuarentena se revisa, la de métricas se grafica

```
-- patrón cuarentena
evaluada = evaluar(df, REGLAS)      -- + columna _fallas

demanda_validada = evaluada WHERE size(_fallas) = 0
demanda_cuarentena = evaluada WHERE size(_fallas) > 0

-- _fallas: ['valor_no_negativo'] ← quién la mandó aquí
```

```
-- calidad_metricas (lab) / event log (pipeline)
regla          accion      fallas  evaluadas
valor_no_nulo   drop        1      145413
valor_no_negativo  cuarentena  1      145413
variable_conocida drop        1      145413
regulado_sin_clasificar warn      1      145413
perdidas_coherentes cuarentena  1      72705
```



Lakehouse Monitoring (clase 15)

Perfila la tabla en el tiempo: distribución, nulos, frescura. Alerta cuando cambian.



SLA de frescura

'Plata refleja toda publicación de XM antes de las 8 a. m.' Se mide con `max(_ingested_at)`; se alerta si no se cumple.

Sector: pérdidas técnicas, no técnicas y el balance de energía

Un dato de pérdidas mal validado se convierte en dinero: entra en la tarifa



Técnicas

Calor en conductores y transformadores. Inevitables; se estiman por modelo de red.



No técnicas

Fraude, errores de medida, errores de facturación. Se reducen con planes que la CREG exige y remunera.



Balance de energía

Lo que entra = lo que se vende + lo que se pierde. En nuestros datos: pérdidas $\leq$ demanda por serie-día. Si falla, no es un dato raro: es medida o liquidación.

Norma	Qué establece	Qué implica para plata
Resolución CREG 015 de 2018	Metodología de remuneración de la distribución; planes de reducción de pérdidas por operador de red	El nivel de pérdidas reconocido es un insumo tarifario: la regla de balance va a cuarentena, no a drop
Resolución CREG 167 de 2020	Ajusta la aplicación de los planes de reducción de pérdidas	Series de pérdidas con historial auditable (bronce) y validado (plata)

Fuente: gestornormativo.creg.gov.co/gestor/entorno/ee_r_perdidas_energia_planes_reduccion_resolucion_15_2018.html

Casos: pipelines declarativos con reglas en la entrada

Casos publicados por el proveedor; los números son verificables en los enlaces de lecturas.md

Empresa	Qué hicieron	Cifras	Lección para plata
Block (Square, Cash App)	Eventos de Kafka → bronce → plata con Spark Declarative Pipelines; regla de calidad en la entrada: todo evento con id válido	≈ 10 pipelines diarios, ≈ 2 TB; +90 % velocidad de desarrollo; de días a horas; 150 pipelines más en camino	Las reglas van lo más cerca posible de la fuente
E.ON (Alemania)	Un data lake para 20+ operadores de distribución; extraer una vez de la fuente y distribuir a muchas aplicaciones	1 TB/día; miles de millones de registros; 120+ aplicaciones integradas	Una plata validada que todos consumen, en vez de cada equipo limpiando lo suyo
SSE Airtricity (Irlanda)	Miles de millones de filas de medidores ingeridas y transformadas cada noche con pipelines declarativos	48 lecturas/día por medidor; 4 meses de idea a producción	El pipeline nocturno es el mismo patrón que hoy, a otra escala

Fuente: databricks.com/customers/block/delta-live-tables · databricks.com/customers/eon · databricks.com/customers/sse-airtricity

Free Edition vs. Azure Databricks: qué simulamos hoy

El código de las reglas es idéntico. La diferencia es quién lo ejecuta y cuántas veces

Elemento	Hoy en Free Edition	En XM sobre Azure
Pipelines activos	Uno por tipo en todo el workspace → la docente corre el pipeline en dev	Uno por ambiente, corrido por el job diario
Lo que hace cada estudiante	Validador por lotes con las mismas REGLAS, en su esquema	No aplica: el pipeline es la única ruta a plata
Métricas	Tabla calidad_metricas escrita por el validador	Event log del pipeline → Log Analytics; dashboard de calidad
Alertas	Ninguna	fail → alerta inmediata; cuarentena > umbral → ticket
Frescura	Se mira a mano	SLA medido con max(_ingested_at); Lakehouse Monitoring (clase 15)

Demo: el pipeline silver_energia en dev

Pipeline creado antes de clase (Jobs & Pipelines → Create → ETL pipeline · notebooks/02_silver_pipeline.py · catálogo dev · esquema silver_energia)

1 Mirar el DAG antes de ejecutar

demanda_vigente → demanda_validada / demanda_cuarentena → balance_serie_dia. Nadie escribió el orden

2 Ejecutar (Start)

Cada nodo muestra filas escritas y filas que fallaron reglas

3 Pestaña Data quality de demanda_validada

Cinco reglas, cero fallas sobre el dato real

4 balance_serie_dia

72.704 serie-días; regla perdidas_coherentes: 0 fallas

```
SQL
-- event log, después de correr
SELECT
  details:flow_progress.data_quality
  .expectations AS reglas
FROM event_log(TABLE(
  dev.silver_energia.demanda_validada))
WHERE event_type = 'flow_progress'
ORDER BY timestamp DESC LIMIT 1;
```

Si alguien más tiene un pipeline corriendo, el nuestro espera: un solo pipeline activo por tipo en Free Edition.

Lab 5 (1/2): las reglas en src y el validador por lotes

clases/c05-calidad-pipelines/notebooks/lab05_calidad_validador · widget usuario · requiere el lab 4 (demanda_raw_vigente)

1 Leer rules.py

REGLAS es la única definición de qué es válido. Cada regla: expresión SQL + acción

2 Implementar evaluar() en validador.py

Agrega `_fallas`: array con los nombres de las reglas que fallaron. Seis líneas

3 Probar con cinco filas sintéticas

Una por regla. La celda de prueba dice cuál falla

4 Validar tu vista vigente

`demanda_validada`, `demanda_cuarentena` y `calidad_metricas` en tu esquema. 0 fallas

```
src/xm_demanda/quality/validador.py
# validador.py — lo que escribes
def evaluar(df, reglas):
    fallas = [
        F.when(~F.expr(expr), F.lit(n))
        for n, (expr, _) in reglas.items()]
    return df.withColumn(
        "_fallas",
        F.array_compact(F.array(*fallas)))

# aplicar() ya viene: separa por acción
```

Error típico: `F.expr(expr)` en vez de `~F.expr(expr)`. `_fallas` debe listar las reglas que NO se cumplen.

Lab 5 (2/2): cinco filas malas, balance, métricas y el pipeline

A dónde va cada fila según su acción, y en qué se diferencia el pipeline del validador

5 Inyectar cinco filas malas y validar

1 a cuarentena · 2 drop · 1 warn · 1 fail (la ves y la quitas)

6 Balance por serie-día

Vista ancha + regla perdidas_coherentes → cuarentena

7 calidad_metricas

regla, acción, fallas, evaluadas, ejecutado_en. El reporte por lotes

8 Leer 02_silver_pipeline.py

Mismas REGLAS. Responde: ¿qué cambia entre validador y pipeline?

9 verificar() · commit · PR

feat(c05): reglas de calidad, validador y cuarentena

Criterios de aceptación

- demanda_validada con 145.408 filas; cuarentena con 2 (valor negativo, balance)
- calidad_metricas con una fila por regla y ejecución
- evaluar() implementada; test_validador.py en verde
- Respuesta escrita sobre validador vs. pipeline
- verificar() en OK; PR en verde

Comparar con s05

```
github.com/<docente>/Databricks-to-XM/compare/  
s05...<tú>:Databricks-to-XM:main · o git diff s05 --  
src/xm_demanda/quality notebooks/02_silver_pipeline.py
```

Así sería en Azure Databricks: pipeline por ambiente y alertas

Hoy	En XM
Un pipeline en dev, corrido a mano por la docente	silver_energia en dev, qa y prod, desplegado por bundle (resources/pipeline_silver.yml) y corrido por el job diario
Validador por lotes en el esquema personal	No existe: el pipeline es la única ruta a plata; el validador queda como herramienta de pruebas locales
calidad_mtricas escrita por el notebook	Event log del pipeline exportado a Log Analytics; dashboard de calidad por regla y por día
Cuarentena se mira en clase	Alerta cuando cuarentena supera un umbral; ticket al dueño del esquema (governance.md)
fail se ve en pantalla	fail → notificación inmediata (Teams / correo) y el job diario se detiene antes de oro
Frescura a ojo	SLA 'plata antes de las 8 a. m.' medido con Lakehouse Monitoring

Lo que no cambia: REGLAS en src, el patrón cuarentena, las métricas por regla.

Resumen de la clase 5 y preparación para la clase 6

Logros de hoy

✓ src es producto, notebook es interfaz

La estructura se elige por lo que corre sin ustedes

✓ Cuándo sí y cuándo no

Análisis de una vez: notebook. Lo corre un job: src + tests + conf

✓ Reglas con acción, en un solo sitio

warn · drop · cuarentena · fail

✓ Pipelines declarativos

Qué tablas existen, no cómo. DAG deducido, métricas por tabla

✓ Balance de energía como regla

pérdidas $\leq$ demanda; a cuarentena, no a drop

Próxima clase

➔ Modelado de plata

Pivote a formato ancho; dimensión CIUU

➔ MERGE idempotente y time travel

Qué versión del dato vio el modelo el día X

➔ Liquid clustering y OPTIMIZE

Rendimiento sin particiones a mano

➔ Traer a la clase 6

Lab 5 terminado: plata se construye sobre demanda_validada



Gracias

Nos vemos en la clase 6: Modelado Silver, de reporte a tabla analítica

DOCENTE

Manuela Larrea Gómez